

EDS Assignment 02

Name: Prathamesh Suresh Tandale

PRN: 202501040272

Batch: CS3-C32

Blog Authorship Corpus - 20 Pandas & NumPy Solutions

This notebook processes the `blogtext.csv` dataset, solving 20 distinct data manipulation problems.

```
In [6]: import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

# Load the actual dataset
df = pd.read_csv('blogtext.csv', on_bad_lines='skip')
print("Dataset loaded successfully! Shape:", df.shape)
```

Dataset loaded successfully! Shape: (681284, 7)

1. Initial Inspection

Load the dataset into a Pandas DataFrame and display the first 5 rows, the data types of each column, and the total memory usage.

```
In [7]: print("--- First 5 rows ---")
print(df.head())
print('\n--- Data Types ---')
print(df.dtypes)
print(f'\nTotal Memory Usage: {df.memory_usage(deep=True).sum()} bytes')
```

```

--- First 5 rows ---
      id gender  age      topic      sign      date \
0  2059027  male   15      Student      Leo  14,May,2004
1  2059027  male   15      Student      Leo  13,May,2004
2  2059027  male   15      Student      Leo  12,May,2004
3  2059027  male   15      Student      Leo  12,May,2004
4  3581210  male   33  InvestmentBanking  Aquarius  11,June,2004

```

```

      text
0  Info has been found (+/- 100 pages,...
1  These are the team members:  Drewe...
2  In het kader van kernfusie op aarde...
3      testing!!!  testing!!!
4  Thanks to Yahoo!'s Toolbar I can ...

```

```

--- Data Types ---

```

```

id      int64
gender   str
age      int64
topic    str
sign     str
date     str
text     str
dtype: object

```

Total Memory Usage: 1047568675 bytes

2. Missing Values

Identify if there are any missing (NaN) or empty string values in the `text` column and remove those rows.

```

In [8]: df['text'] = df['text'].replace('', np.nan)
initial_len = len(df)
df = df.dropna(subset=['text'])
print(f'Rows before: {initial_len} | Rows after dropping missing text: {len(df)}')

```

Rows before: 681284 | Rows after dropping missing text: 681284

3. Unique Authors

Calculate the total number of unique authors (using the `id` column) present in the dataset.

```

In [9]: unique_authors = df['id'].nunique()
print(f'Total unique authors: {unique_authors}')

```

Total unique authors: 19320

4. Categorical Conversion

Convert the `gender`, `topic`, and `sign` columns to Pandas categorical data types to optimize memory usage.

```
In [10]: for col in ['gender', 'topic', 'sign']:
          df[col] = df[col].astype('category')
          print(df[['gender', 'topic', 'sign']].dtypes)
```

```
gender    category
topic     category
sign      category
dtype: object
```

5. Date Parsing

Convert the `date` column into a standardized Pandas datetime format, handling any formatting inconsistencies.

```
In [11]: df['date'] = pd.to_datetime(df['date'], errors='coerce')
          print(df['date'].head())
```

```
0    2004-05-14
1    2004-05-13
2    2004-05-12
3    2004-05-12
4    2004-06-11
Name: date, dtype: datetime64[us]
```

6. Demographic Filtering

Extract a subset of the data containing only blog posts written by users aged 13 to 17.

```
In [12]: teens_df = df[(df['age'] >= 13) & (df['age'] <= 17)]
          print(f'Found {len(teens_df)} posts by teens (13-17).')
          print(teens_df.head(3))
```

Found 235867 posts by teens (13-17).

	id	gender	age	topic	sign	date	\
0	2059027	male	15	Student	Leo	2004-05-14	
1	2059027	male	15	Student	Leo	2004-05-13	
2	2059027	male	15	Student	Leo	2004-05-12	

	text
0	Info has been found (+/- 100 pages,...
1	These are the team members: Drewe...
2	In het kader van kernfusie op aarde...

7. Multi-condition Filtering

Filter the dataset to find all posts written by female authors discussing the 'Technology' or 'Science' topics.

```
In [13]: tech_sci_females = df[(df['gender'] == 'female') & (df['topic'].isin(['Technology', 'Science']))]
print(f'Found {len(tech_sci_females)} posts matching criteria.')
print(tech_sci_females.head(3))
```

Found 9273 posts matching criteria.

	id	gender	age	topic	sign	date	\
1281	3931473	female	26	Science	Sagittarius	2004-07-17	
1282	3931473	female	26	Science	Sagittarius	2004-07-15	
1283	3931473	female	26	Science	Sagittarius	2004-07-14	

	text
1281	Kevin got an email from the sport...
1282	I wish I could sleep all day and ...
1283	urlLink Google Toolbar FAQ Goo...

8. Length-based Filtering

Filter out all 'micro-blogs' by removing rows where the `text` column contains fewer than 10 characters.

```
In [14]: df = df[df['text'].str.len() >= 10]
print(f'Rows remaining after removing micro-blogs: {len(df)}')
```

Rows remaining after removing micro-blogs: 681135

9. Word Count

Create a new column named `post_length` that calculates the total number of words in each blog post using Pandas string methods.

```
In [15]: df['post_length'] = df['text'].str.split().str.len()
print(df[['text', 'post_length']].head(3))
```

	text	post_length
0	Info has been found (+/- 100 pages,...	28
1	These are the team members: Drewe...	20
2	In het kader van kernfusie op aarde...	4326

10. Temporal Features

Extract the 'year' and 'month' from the `date` column and store them in two new separate columns.

```
In [16]: df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
print(df[['date', 'year', 'month']].head(3))
```

	date	year	month
0	2004-05-14	2004.0	5.0
1	2004-05-13	2004.0	5.0
2	2004-05-12	2004.0	5.0

11. Age Bins

Use NumPy's `where` or Pandas' `cut` method to create a new column called `age_group`, categorizing authors into 'Teens' (<20), 'Twenties' (20-29), and 'Adults' (30+).

```
In [17]: bins = [0, 19, 29, 150]
labels = ['Teens', 'Twenties', 'Adults']
df['age_group'] = pd.cut(df['age'], bins=bins, labels=labels)
print(df[['age', 'age_group']].head(5))
```

	age	age_group
0	15	Teens
1	15	Teens
2	15	Teens
3	15	Teens
4	33	Adults

12. Topic Popularity

Determine the top 10 most frequently written about topics in the corpus based on the total number of posts.

```
In [18]: top_topics = df['topic'].value_counts().head(10)
print('Top 10 Topics:\n', top_topics)
```

```
Top 10 Topics:
topic
indUnk      250964
Student     153877
Technology   42029
Arts         32446
Education    29627
Communications-Media  20137
Internet     16001
Non-Profit   14700
Engineering  11650
Law          9034
Name: count, dtype: int64
```

13. Average Post Length

Calculate the mean and median `post_length` for each `gender` category.

```
In [19]: length_stats = df.groupby('gender', observed=False)['post_length'].agg(['mean', 'median'])
print(length_stats)
```

```
          mean  median
gender
female  206.502089   123.0
male    195.308347   103.0
```

14. Age Distribution by Topic

Find the average age of authors for each unique `topic` and sort the results in descending order.

```
In [20]: age_by_topic = df.groupby('topic', observed=False)['age'].mean().sort_values(ascending=False)
print('Average Age by Topic:\n', age_by_topic.head())
```

```
Average Age by Topic:
topic
Transportation  31.415305
Accounting      30.829026
Fashion         29.956298
Construction    29.736505
Publishing      29.478782
Name: age, dtype: float64
```

15. Astrological Activity

Group the data by `sign` and calculate the total number of posts and the maximum `post_length` for each astrological sign.

```
In [21]: astro_activity = df.groupby('sign', observed=False).agg(total_posts=('id', 'count'), max_length=('post_length', 'max'))
print(astro_activity.head())
```

	total_posts	max_length
sign		
Aquarius	49669	131169
Aries	64968	14987
Cancer	65025	22991
Capricorn	49193	28870
Gemini	51979	25436

16. Most Prolific Authors

Identify the top 5 author ids who have written the highest total volume of words across all their posts combined.

```
In [22]: prolific_authors = df.groupby('id')['post_length'].sum().sort_values(ascending=False).head(5)
print('Top 5 Prolific Authors (by total words):\n', prolific_authors)
```

Top 5 Prolific Authors (by total words):

id	
1476382	449321
470861	435065
780903	420393
665500	339047
1151815	308164

Name: post_length, dtype: int64

17. Statistical Variance

Use NumPy to calculate the standard deviation and variance of the `post_length` specifically for posts categorized under the 'Education' topic.

```
In [23]: education_lengths = df[df['topic'] == 'Education']['post_length'].dropna()
std_dev = np.std(education_lengths)
variance = np.var(education_lengths)
print(f'Education posts - Standard Deviation: {std_dev:.2f}')
print(f'Education posts - Variance: {variance:.2f}')
```

Education posts - Standard Deviation: 774.56
Education posts - Variance: 599948.77

18. Pivot Table Analysis

Create a pivot table showing the count of total posts, with `gender` as the index and `age_group` as the columns.

```
In [24]: pivot = df.pivot_table(index='gender', columns='age_group', aggfunc='size', fill_value=0, observed=False)
print(pivot)
```

age_group	Teens	Twenties	Adults
gender			
female	115377	162034	58619
male	120456	159323	65326

19. Temporal Trends

Find the specific year and month combination that saw the highest overall volume of blog posts published.

```
In [25]: temporal_trends = df.groupby(['year', 'month']).size()
if not temporal_trends.empty:
    top_period = temporal_trends.idxmax()
    max_volume = temporal_trends.max()
    print(f'Highest volume was in Year {int(top_period[0])}, Month {int(top_period[1])} with {max_volume} posts.')
```

Highest volume was in Year 2004, Month 7 with 149129 posts.

20. Percentile Analysis

Identify the 90th percentile for `post_length` using NumPy, and filter the DataFrame to show only the posts that exceed this length threshold.

```
In [26]: percentile_90 = np.percentile(df['post_length'].dropna(), 90)
long_posts = df[df['post_length'] > percentile_90]
print(f'90th Percentile Length: {percentile_90:.1f} words')
print(f'Number of posts exceeding this: {len(long_posts)}')
print(long_posts[['text', 'post_length']].head(3))
```

90th Percentile Length: 470.0 words

Number of posts exceeding this: 67963

	text	post_length
2	In het kader van kernfusie op aarde...	4326
5	I had an interesting conversation...	662
11	If you click on my profile you'll...	499